

RESOLUTION NUMERIQUE D'EQUATION DE TYPE $F(X) = 0$

```
import numpy as np
import matplotlib.pyplot as plt

a0=160 #mm
b0=62 #mm

a3=10 #mm
b3=185 #mm

r1=25#mm
l2=72#mm

#reformulation du problème

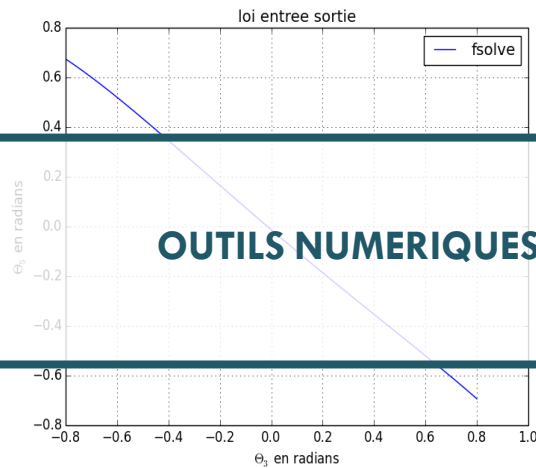
theta=60*np.pi/180

def f(a):
    """fonction définissant la loi E/S de la table à bille
    a: angle table :
    theta : angle moteur"""
    return (l2**2 - (c1*np.cos(theta)-b3*np.cos(a)+a3*np.sin(a)+a0)**2)

#tracé de la courbe f(alpha) pour alpha variant de -pi/2 à pi/2
a_min=-10*np.pi/180
a_max=10*np.pi/180
l2=np.linspace(a_min,a_max,100)
l2=[]
for alpha in l2:
    f.append(f(alpha))
plt.figure('trace de f(alpha)')
plt.plot(l2,f)

plt.grid()
#on observe que la courbe coupe l'axe des abscisses par alpha=0

#reformulation f(alpha)=0 par dichotomie
def dichotomie(f, a, b, epsilon):
    """résolution de l'équation f(x)=0 par la méthode de dichotomie
    f : fonction
    a : borne inférieure
    b : borne supérieure
    epsilon : précision"""
    m=(a+b)/2
    if f(m)>0:
        a=m
    else:
        b=m
    return m
```



OUTILS NUMERIQUES

SYNTHESE

Compétences Visées :

- Résoudre numériquement une équation ou un système d'équations (réécriture des équations d'un problème, résolution de problèmes du type $f(x)=0$, intégration et dérivation numérique)

1 REECRITURE DU PROBLEME

Les méthodes de ce chapitre permettent de résoudre des équations du type $f(x)=0$.

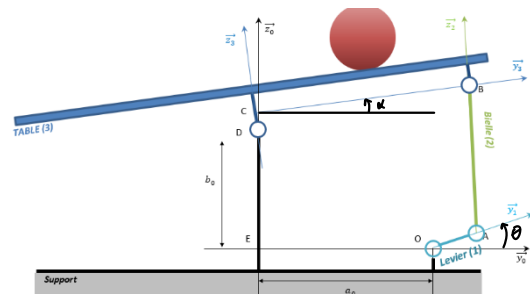
Les équations doivent être donc auparavant être réécrites sous cette forme.

Exemple : BBT

OBJECTIF : déterminer la loi E/S du système de transformation de mouvement :

Paramètre d'entrée : θ : angle du moteur (levier)

Paramètre de sortie : α : angle de la table



L'écriture de la fermeture géométrique conduit à la relation suivante :

$$(a_0 + r_1 \cos \theta - b_3 \cos \alpha + a_3 \sin \alpha)^2 + (r_1 \sin \theta - b_3 \sin \alpha - a_3 \cos \alpha - b_0)^2 = l_2^2 \quad (1)$$

$a_0, r_1, a_3, b_3, l_2, b_0$ sont les dimensions des pièces, constantes et connues.

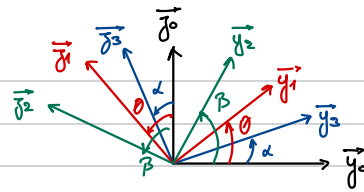
On décide de déterminer α pour chaque valeur de θ .

A chaque pas de calcul, la valeur de θ est fixée ainsi que tous les autres paramètres constants.

Réécrire l'équation (1) pour avoir une équation de la forme : $f(\alpha) = 0$

$$\vec{EO} + \vec{OA} + \vec{AB} + \vec{BC} + \vec{CD} + \vec{DE} = \vec{0}$$

$$a_0 \vec{y}_0 + x_1 \vec{y}_1 + l_2 \vec{z}_2 - l_3 \vec{y}_3 - a_3 \vec{z}_3 - b_0 \vec{z}_0 = \vec{0}$$



projete :

$$y_0 \begin{cases} a_0 + x_1 \cos \theta - l_2 \sin \beta - b_3 \cos \alpha + a_3 \sin \alpha = 0 \\ x_1 \sin \theta + l_2 \cos \beta - b_3 \sin \alpha - a_3 \cos \alpha - b_0 = 0 \end{cases}$$

$$\text{Eliminons } \beta: l_2^2 (\cos^2 \beta + \sin^2 \beta) = (a_0 + x_1 \cos \theta - b_3 \cos \alpha + a_3 \sin \alpha)^2 + (b_3 \sin \alpha + a_3 \cos \alpha + b_0 - x_1 \sin \theta)^2$$

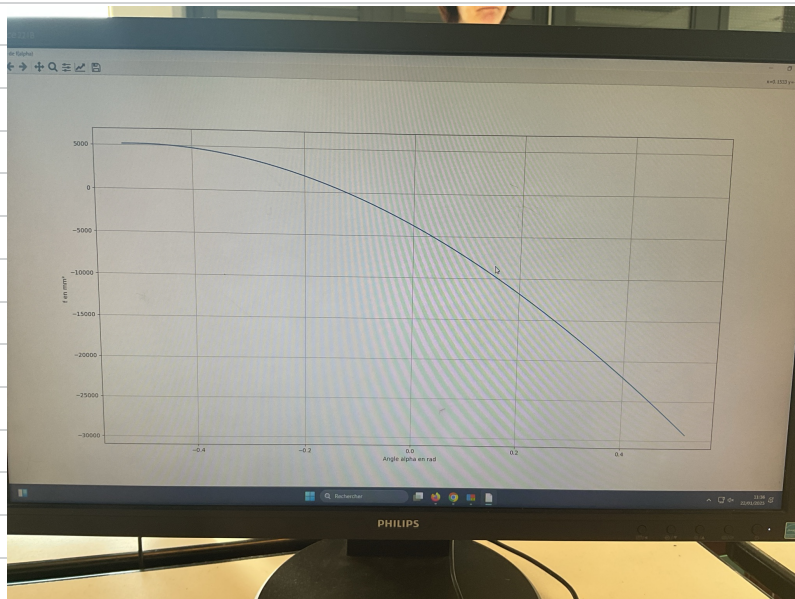
$$l_2^2 = (a_0 + x_1 \cos \theta - b_3 \cos \alpha + a_3 \sin \alpha)^2 + (x_1 \sin \theta - b_3 \sin \alpha + a_3 \cos \alpha - b_0)^2$$

$$f(\alpha) = 0$$

$$\hookrightarrow f(\alpha) = (a_0 + x_1 \cos \theta - b_3 \cos \alpha + a_3 \sin \alpha)^2 + (x_1 \sin \theta - b_3 \sin \alpha + a_3 \cos \alpha - b_0)^2 - l_2^2$$

Pour Newton il faut la dérivée.

$$f'(\alpha) = 2(a_0 + x_1 \cos \theta - b_3 \cos \alpha + a_3 \sin \alpha) \cdot (b_3 \sin \alpha + a_3 \cos \alpha) + 2(x_1 \sin \theta - b_3 \sin \alpha + a_3 \cos \alpha - b_0) \cdot (a_3 \sin \alpha - b_3 \cos \alpha)$$



f s'annule en $-0,12 \text{ rad}$

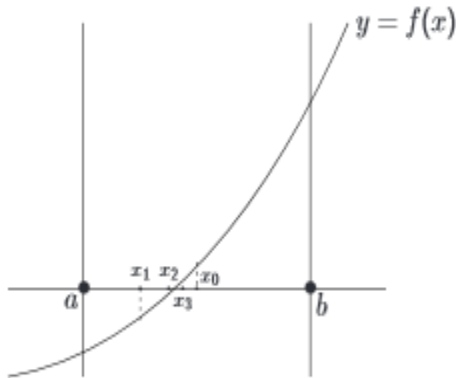
Dichotomie $\Rightarrow -0,12268402286104188$ en 34 itérations

Newton $\Rightarrow -0,12268402286032248$ en 4 itérations

2 METHODE DE DICHOTOMIE

On considère un intervalle $[a, b]$ et une fonction f continue de $[a, b]$ dans $\mathbb{R}$. On suppose que $f(a) \times f(b) < 0$ et que l'équation $f(x) = 0$ admet une unique solution α sur l'intervalle $[a, b]$.

La méthode de dichotomie consiste à construire une suite (x_n) qui converge vers α de la manière suivante :



Dichotomie

Initialisation : on prend pour x_0 le milieu de $[a, b]$. La racine se trouve alors dans l'un des deux intervalles $[a, x_0]$ ou $]x_0, b]$.

- Si $f(a) \times f(x_0) \leq 0$, alors $\alpha \in [a, x_0]$.

On pose alors $a_1 = a$ et $b_1 = x_0$.

- Sinon, $f(a) \times f(x_0) > 0$, alors $\alpha \in]x_0, b]$.

On pose $a_1 = x_0$ et $b_1 = b$.

On prend alors pour x_1 le milieu de $[a_1, b_1]$. On construit ainsi une suite x_n :

$$x_0 = \frac{a+b}{2}; x_1 = \frac{a_1+b_1}{2}; x_2 = \frac{a_2+b_2}{2}; \text{ telle que } |\alpha - x_n| \leq \frac{b-a}{2^{n+1}}$$

Etant donnée une précision ε , cette méthode permet d'approcher α en un nombre prévisible d'itérations.

Algorithme



```
def Dichotomie (f, a : float, b : float, epsilon : float) :
```

```
    while abs(b-a) > 2 * epsilon :
```

```
        m = (b+a)/2
```

```
        if f(a) * f(m) >= 0 :
```

```
            a = m
```

```
        else :
```

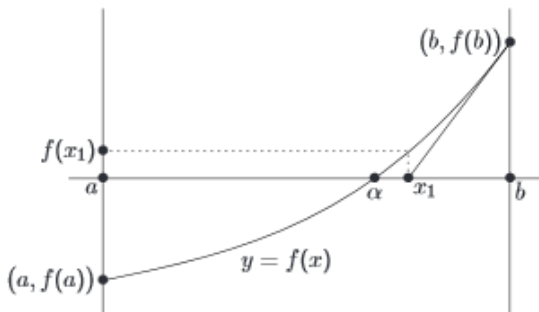
```
            b = m
```

```
    return (a+b)/2
```

3 METHODE DE NEWTON

La méthode de Newton consiste à remplacer la courbe par sa tangente en une de ses deux extrémités. Le point x_1 est l'intersection de cette tangente avec l'axe Ox .

Pour faire le dessin, on va se placer dans le cas où $f' > 0$ et $f'' < 0$. On prend alors $x_0 = b$.



Méthode de Newton

Traçons la tangente à la courbe représentative de f passant par $(b, f(b))$. L'équation de cette tangente est :

$$y(x) = f'(b) * (x - b) + f(b)$$

Son intersection avec l'axe des abscisses a une ordonnée nulle ($y(x_1) = 0$) et son abscisse vaut alors $x_1 = b - \frac{f(b)}{f'(b)}$.

On trace ensuite la tangente à la courbe au point $(x_1, f(x_1))$. Le réel x_2 est l'abscisse de l'intersection de cette deuxième tangente avec l'axe des abscisses et on réitère ce procédé.

Remarque – Si on prenait la tangente à la courbe en $(a, f(a))$, son intersection avec l'axe des abscisses ne serait pas, pour l'exemple du dessin, dans l'intervalle $[a, b]$.

Terminaison

On arrête l'algorithme lorsque l'écart entre deux termes successifs est « petit », autrement dit lorsque $|x_{n+1} - x_n| \leq \varepsilon$.

Algorithme

```
def Newton (f, df, x0, epsilon) :
```

```
    x = x0
```

```
    x1 = x - f(x) / df(x)
```

```
    i, imax = 0, 60
```

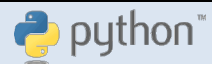
```
    while abs(x - x1) > epsilon and i < imax :
```

```
        x = x1
```

```
        x1 = x - f(x) / df(x)
```

```
        i = i + 1
```

```
    return (x1, i)
```



4 FONCTION PREDEFINIE DANS SCIPY

La bibliothèque **scipy.optimize** contient la fonction **newton** qui implémente cet algorithme.

Exemple (pour les fonctions f et df définies auparavant)



```
import scipy.optimize as spo

x0 = 2    # x0 est la valeur de départ

S=spo.newton(f, x0, df) # cette fonction renvoie la valeur approchée de la racine à 1.48e-08
```

Si la dérivée n'est pas connue, on peut utiliser aussi

```
S=spo.newton(f, x0)
```

L'algorithme utilise alors la méthode des sécantes pour approximer la dérivée. On obtiendra un résultat un peu moins précis.

5 COMPARAISON DES METHODES

Vitesse de convergence

On note $\delta_n = |\alpha - x_n|$,

δ_n représente l'écart entre la solution α du problème et x_n la solution retournée par l'algorithme au bout de l'étape n .

Dichotomie	Newton
Linéaire	Quadratique
$\exists K \in \mathbb{R}_+, \forall n \in \mathbb{N}, \delta_{n+1} \leq K \delta_n$	$\exists K \in \mathbb{R}_+, \forall n \in \mathbb{N}, \delta_{n+1} \leq K \delta_n^2$

La méthode de Newton est donc plus rapide.

Terminaison

La question est de savoir si l'algorithme se termine, et s'il renvoie bien la valeur du zéro...

On peut montrer facilement que **la terminaison est assurée avec la méthode de dichotomie**.

C'est plus compliqué avec la méthode de Newton : la suite des x_n n'est pas nécessairement convergente et risque de s'éloigner du zéro de f . On peut montrer néanmoins que si f' et f'' gardent un signe constant sur l'intervalle (fonction strictement monotone et de même concavité), alors la suite converge bien vers le zéro. En partant d'une valeur x_0 assez proche de la solution cette condition sera vérifiée.

Il est donc conseillé d'utiliser la méthode de **Newton en partant d'une valeur x_0 assez proche de la solution**.